



UNIVERSIDAD DE LAS AMÉRICAS PUEBLA

DEPARTAMENTO DE COMPUTACIÓN, ELECTRÓNICA Y MECATRÓNICA
LIS3082-2 — INTELIGENCIA ARTIFICIAL

Agente Neuro-Simbólico para Clasificación de Insectos mediante YOLO, MobileNet y DeepProbLog

Proyecto Final

Estudiantes:

Bruno Tarango Garay - 182639
Miguel Ángel García Arrieta - 181266
Diego Flores Martínez - 179921
Víctor Hugo de la Calleja Mojica - 181626
Ingeniería en Sistemas Computacionales

Profesor:

Edwin Montes Orozco

SAN ANDRÉS CHOLULA, PUEBLA. PRIMAVERA 2026

Índice

1. Problema y objetivo	2
2. Definición del agente	2
2.1. Función de decisión	2
3. Dataset y preparación de datos	3
3.1. Estructura original y limpieza de datos	3
3.2. Jerarquía taxonómica y selección de especies	4
3.3. Composición del conjunto de imágenes y anotaciones	4
3.4. Distribución final del dataset	6
4. Implementación	7
4.1. YOLO	7
4.2. MobileNet	8
4.3. Interfaz JSON y DeepProbLog	8
5. Experimentos realizados	9
5.1. Resultados de YOLO	10
5.2. Resultados de MobileNet	10
5.3. Resultados del agente completo	10
6. Análisis del comportamiento	11
7. Limitaciones	12
8. Conclusiones	12
9. Enlaces de entrega	12

1. Problema y objetivo

El problema consiste en clasificar insectos a partir de imágenes usando un agente que no dependa solamente de una red neuronal. La clasificación puede ser difícil porque varias especies comparten rasgos visuales, algunas partes del cuerpo no siempre son visibles y las predicciones de visión pueden tener errores.

El objetivo del proyecto fue construir un agente capaz de recibir una imagen, extraer evidencia visual mediante modelos supervisados y producir una decisión final mediante razonamiento lógico-probabilístico. Para esto se integraron tres componentes: YOLO, MobileNet y DeepProbLog.

En la arquitectura implementada, YOLO y MobileNet no forman una cadena directa. Ambos modelos procesan la imagen de entrada, pero producen evidencias distintas. YOLO aporta detecciones de partes visibles del insecto, mientras que MobileNet aporta una predicción global de especie y rasgos morfológicos. Después, ambas salidas se combinan en archivos JSON para que DeepProbLog realice la inferencia final.

2. Definición del agente

El sistema se considera un agente porque recibe un estado s , evalúa posibles decisiones y genera una acción a . En este proyecto, el estado corresponde a la información extraída de una imagen de insecto:

$$s = \{imagen, prediccion_mobilenet, rasgos, partes_detectadas\}$$

La acción del agente es la clase final seleccionada. Las clases consideradas fueron: mosquito tigre asiático, abeja, mariposa monarca, catarina, saltamontes y mantis religiosa.

2.1. Función de decisión

El agente toma una imagen de un insecto y debe decidir la clase final más probable. La decisión se define como:

$$a^* = \arg \max_a f(s, a)$$

donde s representa el estado actual del sistema, a representa una posible acción o especie candidata, y $f(s, a)$ es la función que evalúa qué tan conveniente es elegir esa especie con base en la evidencia disponible.

De forma conceptual, la función de decisión puede representarse como:

$$f(s, a) = w_1 P_{cnn}(a) + w_2 P_{traits}(a) + w_3 R_{logic}(a) + w_4 Q_{image}$$

En esta expresión, $P_{cnn}(a)$ representa la probabilidad asignada por el modelo visual a una especie candidata, $P_{traits}(a)$ mide la compatibilidad entre los rasgos morfológicos y esa especie, $R_{logic}(a)$ representa el soporte de las reglas de DeepProbLog, y Q_{image} representa la calidad o visibilidad de la evidencia visual. Los pesos w_1 , w_2 , w_3 y w_4 indican la importancia relativa de cada factor.

En la implementación del proyecto, esta función no se programó como una suma numérica exacta. Se implementó como una lógica de decisión basada en confianza, soporte visual y reglas simbólicas. Por eso, el agente no toma la decisión únicamente con la clase predicha por MobileNet. En cambio, combina la predicción visual, los rasgos morfológicos, las partes detectadas por

YOLO y las reglas de DeepProbLog para seleccionar la clase más probable o marcar el caso como revisión.

En la implementación final, DeepProbLog funciona como una capa de razonamiento sobre evidencia generada previamente por YOLO y MobileNet. Es decir, no recibe imágenes crudas, sino hechos estructurados obtenidos desde archivos JSON.

3. Dataset y preparación de datos

El conjunto de datos se construyó a partir de un listado inicial de especies y se enriqueció mediante anotación manual, verificación taxonómica y organización jerárquica para permitir el entrenamiento de los componentes YOLO y MobileNet. En esta sección se describe la estructura original del dataset, los criterios de selección de especies, la jerarquía de clasificación utilizada, la composición de las imágenes y el particionado final para entrenamiento, validación y prueba.

3.1. Estructura original y limpieza de datos

Como punto de partida se dispuso de un listado general de insectos que contenía nombres científicos de especies candidatas. Este listado fue depurado eliminando aquellas especies que correspondían a ejemplares con pocas imágenes disponibles o con problemas de etiquetado. Posteriormente, se utilizó un registro de nombres comunes y clasificaciones taxonómicas para unificar criterios de nomenclatura y definir la jerarquía de órdenes y familias. Finalmente, se generó una tabla de evidencia que asocia cada imagen con su ruta de archivo, nombre común, orden y familia. Esta tabla fue fundamental para garantizar la trazabilidad entre las imágenes crudas y las anotaciones posteriores.

La Figura 1 muestra el listado de especies consideradas, indicando cuáles fueron retenidas para el entrenamiento y cuáles fueron descartadas.

Nombre 
 Aedes albopictus (Skuse)
 Anax strenuus Hagen (YA NO)
 Apis mellifera Linnaeus
 Chrysoperla plorabunda (Fitch) (YA NO)
 Coccinella septempunctata Linnaeus
 Danaus plexippus (Linnaeus)
 Dissosteira carolina (Linnaeus)
 Halyomorpha halys (Stål) (YA NO)
 Mantis religiosa Linnaeus

Figura 1: Listado de especies candidatas: las marcadas como *YA NO* fueron descartadas por falta de imágenes o problemas de etiquetado.

3.2. Jerarquía taxonómica y selección de especies

Cada especie pertenece a un orden y familia diferentes, lo que maximiza la diversidad morfológica y reduce el riesgo de confusiones visuales entre clases cercanas. Esta diversidad es esencial para evaluar la capacidad del agente de distinguir insectos basándose tanto en rasgos visuales como en reglas simbólicas.

La Figura 2 muestra la tabla de nombres comunes, nombres científicos, órdenes y familias utilizada para unificar la nomenclatura del dataset.

	A	B	C	D	E	F	G
1	common_name	scientific_name	order	family	genus	species	
2	abbreviated wire	Hypnoidus abbreviatus	Coleoptera	Elateridae	Hypnoidus	abbreviatus	
3	acacia psyllid	Acizzia uncatoides	Hemiptera	Psyllidae	Acizzia	uncatoides	
4	achemon sphinx	Eumorpha acheron	Lepidoptera	Sphingidae	Eumorpha	achemon	
5	acuminate scale	Kilifia acuminata	Hemiptera	Coccidae	Kilifia	acuminata	
6	acute-angled fur	Cryptophagus acutangulus	Coleoptera	Cryptophagidae	Cryptophagus	acutangulus	
7	aerial yellowjack	Dolichovespula areolaris	Hymenoptera	Vespidae	Dolichovespula	arenaria	
8	African fig fly	Zaprionus indianus	Diptera	Drosophilidae	Zaprionus	indianus	
9	African mole cricket	Gryllotalpa africana	Orthoptera	Gryllotalpidae	Gryllotalpa	africana	
10	ailanthus webworm	Atteva punctella	Lepidoptera	Yponomeutidae	Atteva	punctella	
11	alder bark beetle	Alniphagus aspericollis	Coleoptera	Curculionidae	Alniphagus	aspericollis	
12	alder flea beetle	Altica ambiens	Coleoptera	Chrysomelidae	Altica	ambiens	
13	alder spittlebug	Clastoptera obtusa	Hemiptera	Cercopidae	Clastoptera	obtusa	
14	alfalfa blotch leaf miner	Agromyza frontella	Diptera	Agromyzidae	Agromyza	frontella	
15	alfalfa caterpillar	Colias eurytheme	Lepidoptera	Pieridae	Colias	eurytheme	
16	alfalfa gall midge	Asphondylia websteri	Diptera	Cecidomyiidae	Asphondylia	websteri	
17	alfalfa leafcutting wasp	Megachile rotundata	Hymenoptera	Megachilidae	Megachile	rotundata	
18	alfalfa leaf-tier	Dichomeris acuminata	Lepidoptera	Gelechiidae	Dichomeris	acuminata	
19	alfalfa looper	Autographa californica	Lepidoptera	Noctuidae	Autographa	californica	

Figura 2: Tabla taxonómica del dataset: nombre común, nombre científico, orden, familia, género y especie de cada ejemplar incluido.

3.3. Composición del conjunto de imágenes y anotaciones

El dataset contiene únicamente imágenes etiquetadas con una sola especie por imagen. Todas las imágenes fueron anotadas manualmente con cajas delimitadoras (*bounding boxes*) para seis partes del insecto: *antennae*, *body*, *insect* (cuerpo completo), *legs*, *mouthpart* y *wings*.

La Figura 3 muestra la organización del subdirectorio de imágenes por especie dentro del dataset.

Nombre 
 img_000001.jpg 
 img_000280.jpg 
 img_000281.jpg 
 img_000282.jpg 
 img_000283.jpg 
 img_000284.jpg 
 img_000285.jpg 
 img_000286.jpg 
 img_000287.jpg 

Figura 3: Organización del directorio de imágenes: cada especie cuenta con su propio subdirectorío con las imágenes numeradas secuencialmente.

La Figura 4 presenta la tabla de evidencia generada, que asocia cada imagen con su ruta de archivo, nombre común, orden y familia para garantizar la trazabilidad entre imágenes crudas y anotaciones.

image_id	file_path	common_name	order	family
img_000001	Dataset/Aedes albopictus (Skuse)/img_000001.jpg	Asian tiger mosquito	Diptera	Culicidae
img_000002	Dataset/Coccinella septempunctata Linnaeus/img_000002.jpg	sevenspotted lady beetle	Coleoptera	Coccinellidae
img_000003	Dataset/Coccinella septempunctata Linnaeus/img_000003.jpg	sevenspotted lady beetle	Coleoptera	Coccinellidae
img_000004	Dataset/Coccinella septempunctata Linnaeus/img_000004.jpg	sevenspotted lady beetle	Coleoptera	Coccinellidae
img_000005	Dataset/Coccinella septempunctata Linnaeus/img_000005.jpg	sevenspotted lady beetle	Coleoptera	Coccinellidae
img_000006	Dataset/Coccinella septempunctata Linnaeus/img_000006.jpg	sevenspotted lady beetle	Coleoptera	Coccinellidae
img_000007	Dataset/Coccinella septempunctata Linnaeus/img_000007.jpg	sevenspotted lady beetle	Coleoptera	Coccinellidae
img_000008	Dataset/Coccinella septempunctata Linnaeus/img_000008.jpg	sevenspotted lady beetle	Coleoptera	Coccinellidae
img_000009	Dataset/Coccinella septempunctata Linnaeus/img_000009.jpg	sevenspotted lady beetle	Coleoptera	Coccinellidae
img_000010	Dataset/Coccinella septempunctata Linnaeus/img_000010.jpg	sevenspotted lady beetle	Coleoptera	Coccinellidae

Figura 4: Tabla de evidencia del dataset: cada fila asocia el identificador de imagen, la ruta del archivo, el nombre común, el orden y la familia taxonómica.

La Figura 5 muestra los rasgos morfológicos definidos por especie, los cuales fueron utilizados como etiquetas para el entrenamiento multi-salida de MobileNet.

1	Mosquito							
2	wing_count	forewing_type	mouthpart_type	antenna_type	antenna_type	leg_specializati	body_shape	waist_shape
3	one_pair	membranous	piercing_sucking	plumose	filiform	generalist	slender	no_waist
4								
5								
6	Honey Bee							
7	wing_count	forewing_type	mouthpart_type	antenna_type	antenna_type	leg_specializati	body_shape	waist_shape
8	two_pairs	membranous	chewing	filiform	NA	generalist	elongated	narrow_waist
9								
10	Monarch Butterfly							
11	wing_count	forewing_type	mouthpart_type	antenna_type	antenna_type	leg_specializati	body_shape	waist_shape
12	two_pairs	scaly	siphoning	clavate	NA	generalist	slender	no_waist
13								
14	Lady Beetle							
15	wing_count	forewing_type	mouthpart_type	antenna_type	antenna_type	leg_specializati	body_shape	waist_shape
16	two_pairs	elytra	chewing	clavate	NA	generalist	rounded_domed	no_waist
17								
18	Green lacewing							
19	wing_count	forewing_type	mouthpart_type	antenna_type	antenna_type	leg_specializati	body_shape	waist_shape
20	two_pairs	membranous	chewing	filiform	NA	generalist	slender	no_waist
21								
22	American cockroach							
23	wing_count	forewing_type	mouthpart_type	antenna_type	antenna_type	leg_specializati	body_shape	waist_shape
24	two_pairs	tegmina	chewing	filiform	NA	running	oval_flat	no_waist
25								
26	Grasshopper							
27	wing_count	forewing_type	mouthpart_type	antenna_type	antenna_type	leg_specializati	body_shape	waist_shape
28	two_pairs	tegmina	chewing	filiform	NA	jumping	elongated	no_waist
29								
30	Praying Mantis							
31	wing_count	forewing_type	mouthpart_type	antenna_type	antenna_type	leg_specializati	body_shape	waist_shape
32	two_pairs	tegmina	chewing	filiform	NA	grasping	elongated	no_waist
33								
34	Stink bug							
35	wing_count	forewing_type	mouthpart_type	antenna_type	antenna_type	leg_specializati	body_shape	waist_shape
36	two_pairs	hemelytra	piercing_sucking	filiform	NA	generalist	shield_like	no_waist
37								
38	Dragonfly							
39	wing_count	forewing_type	mouthpart_type	antenna_type	antenna_type	leg_specializati	body_shape	waist_shape
40	two_pairs	membranous	chewing	setaceous	NA	generalist	slender	no_waist
41								

Figura 5: Rasgos morfológicos por especie: número de alas, tipo de ala, aparato bucal, tipo de antena, especialización de patas, forma del cuerpo y cintura.

Esta composición permite que YOLO aprenda a localizar las partes del insecto, mientras que MobileNet recibe la imagen completa para clasificar especie y rasgos.

3.4. Distribución final del dataset

Una vez completadas las anotaciones, se realizó un particionado estratificado para mantener la proporción de cada especie en los subconjuntos de entrenamiento, validación y prueba. La Tabla 1 resume la cantidad de imágenes y archivos de etiquetas en cada partición.

Tabla 1: Distribución del dataset por partición.

Partición	Imágenes	Archivos de etiquetas (.txt)
Entrenamiento (Train)	231	231
Validación (Valid)	66	66
Prueba (Test)	34	34

El tamaño del dataset es limitado, lo que impone restricciones importantes. Las clases pequeñas como **mouthpart** y **legs** resultan más difíciles de detectar para YOLO debido a que ocupan

pocos píxeles en la imagen. Además, la variabilidad de fondos, poses e iluminación no cubre todos los escenarios reales, por lo que el sistema puede presentar errores ante imágenes con baja calidad, ángulos poco comunes o partes del insecto parcialmente ocultas. A pesar de estas limitaciones, el dataset es suficiente para demostrar la viabilidad del agente neuro-simbólico y para evaluar el comportamiento de los componentes de visión y razonamiento.

Nombre 
 Aedes albopictus -Skuse-Sin aumentacion.zip 
 apis-mellifera-linnaeus.yolov8.zip 
 Coccinella septempunctata Linnae Sin aumentacion.... 
 Danaus plexippus -Linnaeus sin aumentacion.zip 
 dissosteira-carolina-linnaeus-yolov8.zip 
 mantis-religiosa-linnaeus.yolov8.zip 

Figura 6: Archivos ZIP generados por especie tras el proceso de anotación, listos para integrarse al pipeline de entrenamiento de YOLO y MobileNet.

4. Implementación

La arquitectura real implementada usa dos ramas visuales en paralelo. La primera rama usa YOLO para detectar partes visibles del insecto. La segunda rama usa MobileNet para analizar la imagen completa y predecir la especie junto con rasgos morfológicos. Posteriormente, ambas salidas se integran en un JSON y DeepProbLog usa esa evidencia estructurada para tomar una decisión final.

Imagen $\rightarrow$ YOLO $\rightarrow$ partes detectadas

Imagen $\rightarrow$ MobileNet $\rightarrow$ especie y rasgos morfológicos

partes + especie + rasgos $\rightarrow$ JSON $\rightarrow$ DeepProbLog $\rightarrow$ decisión final

Es importante aclarar que MobileNet no recibe recortes generados por YOLO. Ambos modelos procesan la imagen de entrada, pero generan evidencias distintas. YOLO aporta evidencia localizada sobre partes visibles del insecto, mientras que MobileNet aporta una predicción global de especie y rasgos morfológicos. Después, ambas salidas se integran en un archivo JSON para que DeepProbLog realice el razonamiento final.

Espacio para imagen: images/system_overview.png

Figura 7: Flujo general del agente neuro-simbólico.

4.1. YOLO

YOLO fue entrenado para detectar seis partes del insecto: antenas, cuerpo, insecto completo, patas, aparato bucal y alas. Su función dentro del sistema es aportar evidencia localizada. Es

decir, permite saber qué partes del insecto fueron detectadas en la imagen y con qué frecuencia aparecen.

El mejor modelo fue guardado como:

```
final_models/best_yolo_insect_parts.pt
```

Las clases detectadas por YOLO fueron:

- antennae
- body
- insect
- legs
- mouthpart
- wings

4.2. MobileNet

MobileNet fue usado como modelo supervisado multi-salida. En esta implementación, MobileNet trabaja con la imagen completa del insecto. No recibe como entrada las regiones detectadas por YOLO; su función es producir una predicción global de especie y rasgos morfológicos.

Además de predecir la especie, MobileNet predice rasgos visuales como número de alas, tipo de ala, aparato bucal, tipo de antena, especialización de patas, forma del cuerpo y cintura. El mejor modelo fue guardado como:

```
final_models/best_mobilenet_insect_multitask.pth
```

Los rasgos predichos por MobileNet fueron:

- wing_count
- forewing_type
- mouthpart_type
- antenna_type
- leg_specialization
- body_shape
- waist_shape

4.3. Interfaz JSON y DeepProbLog

El tercer Colab carga los modelos entrenados y genera archivos JSON por imagen. Para cada imagen, se ejecutan dos ramas: YOLO detecta partes visibles y MobileNet predice especie y rasgos usando la imagen completa. Luego, el JSON combina ambas salidas en una estructura común para DeepProbLog.

Los JSONs de prueba se guardan en:

```
ModeloEntrenado/predictions/test_jsons/
```

La integración entre el sistema visual y la capa simbólica se hizo mediante dos archivos principales: `run_batch_integration.py`, encargado de leer los JSONs y generar hechos, y `json_multiclass_rules.pl`, donde se definieron las reglas simbólicas. El flujo fue: leer JSON, transformar la evidencia a hechos, unir esos hechos con las reglas, ejecutar consultas y guardar resultados.

Tabla 2: Conversión de JSON a hechos lógicos.

Campo JSON	Hecho generado	Función
image_id	identificador	Relaciona los hechos con una imagen específica.
predicted_insect	cnn_insect/3	Guarda la especie propuesta por MobileNet y su confianza.
predicted_traits	trait/4	Representa los rasgos morfológicos y su confianza.
part_counts	part_summary/4	Resume partes detectadas por YOLO.
count>0	part_seen/2	Marca presencia visual de una parte del insecto.

Sobre esos hechos se implementaron reglas multiclase. Primero se valida si el caso tiene confianza suficiente con `valid_case/1`. Después se revisa soporte visual mínimo con `visual_support/1`. Luego se activan firmas morfológicas por clase, como `signature_mosquito/1`, `signature_bee/1`, `signature_grasshopper/1`, `signature_mantis/1`, `signature_butterfly/1` y `signature_lady_beetle/1`. Finalmente, el sistema infiere `final_class/2` y, si la especie propuesta es consistente, `final_species/2`. Si la evidencia no es suficiente, se activa `needs_review/1`.

Tabla 3: Reglas principales de la capa simbólica.

Regla	Función
<code>valid_case/1</code>	Verifica que MobileNet tenga confianza mínima aceptable.
<code>visual_support/1</code>	Comprueba que YOLO haya detectado al menos el insecto.
<code>signature_*/1</code>	Evalúa si los rasgos coinciden con una clase simbólica.
<code>final_class/2</code>	Produce la clase final con evidencia suficiente.
<code>final_species/2</code>	Valida que la especie sea consistente con la clase inferida.
<code>needs_review/1</code>	Marca revisión manual si no hay evidencia suficiente.

Esta implementación no reentrenó YOLO ni MobileNet. DeepProbLog fue usado como una capa de razonamiento sobre evidencia ya calculada. Por eso, el sistema funciona como una integración neuro-simbólica modular, pero no como entrenamiento neuro-simbólico de extremo a extremo.

5. Experimentos realizados

Se realizaron tres evaluaciones principales. Primero, se evaluó YOLO con métricas de detección. Segundo, se evaluó MobileNet con accuracy por especie y por rasgo. Finalmente, se evaluó el agente completo usando JSONs procesados por DeepProbLog.

5.1. Resultados de YOLO

Tabla 4: Resultados generales de YOLO en validación.

Modelo	Precision	Recall	mAP50	mAP50-95
YOLOv8n	0.682	0.545	0.591	0.353

Las mejores detecciones se observaron en clases grandes o visualmente claras, como `insect`, `body` y `wings`. Las clases pequeñas, como `mouthpart` y `legs`, tuvieron menor desempeño.

5.2. Resultados de MobileNet

Tabla 5: Resultados de MobileNet en test.

Salida evaluada	Accuracy
Insect label	0.971
Wing count	0.971
Forewing type	1.000
Mouthpart type	0.971
Antenna type	1.000
Leg specialization	0.971
Body shape	0.971
Waist shape	1.000

MobileNet obtuvo un desempeño alto en test. Esto indica que el modelo aprendió patrones visuales útiles para clasificar las especies y sus rasgos principales. Sin embargo, este resultado corresponde al análisis de la imagen completa, no a recortes generados por YOLO.

5.3. Resultados del agente completo

El agente completo se evaluó sobre 34 archivos JSON de prueba. En cada caso, el pipeline ejecutó cinco pasos: lectura del JSON, generación de hechos, unión con reglas, ejecución de consultas y registro del resultado final.

Tabla 6: Resultados globales del agente completo.

Métrica	Valor
Casos evaluados	34
Accuracy de clase	0.8824
Accuracy de especie	0.8824
Macro precision	0.9762

La precisión por clase fue alta en la mayoría de las categorías. Cinco clases alcanzaron precisión de 1.0000. La clase con menor precisión fue `grasshopper`, con 0.8571, lo que muestra que todavía existen confusiones entre firmas morfológicas cercanas.

Tabla 7: Precisión por clase del agente completo.

Clase	Precisión
mosquito	1.0000
bee	1.0000
grasshopper	0.8571
mantis	1.0000
butterfly	1.0000
lady beetle	1.0000

Tabla 8: Casos representativos del agente completo.

Caso	Esperada	MobileNet	DeepProbLog	Interpretación
aedes_albopictus_000280	mosquito	mosquito	mosquito	Caso correcto con <code>signature_mosquito</code> y soporte visual suficiente.
apis_mellifera_000123	bee	bee	bee	Caso correcto con rasgos compatibles y <code>signature_bee</code> .
aedes_albopictus_000293	mosquito	mosquito	review	La confianza o soporte no fue suficiente y se activó <code>needs_review</code> .
mantis_religiosa_000199	mantis	mantis	grasshopper	Error por activación de una firma morfológica incorrecta.

La accuracy final del agente se calcula como:

$$Accuracy = \frac{\text{predicciones correctas}}{\text{total de pruebas}}$$

Estos resultados muestran que el agente no se limitó a repetir la etiqueta de MobileNet. La capa simbólica añadió una validación de consistencia. Cuando la evidencia visual y los rasgos coincidieron, la clasificación final fue correcta en la mayoría de los casos. Cuando no existió suficiente evidencia, el sistema pudo marcar revisión.

6. Análisis del comportamiento

El agente responde a cambios en la entrada porque cada imagen genera un estado diferente. Su comportamiento puede describirse como un filtro de consistencia sobre la salida del módulo visual. La decisión final no depende de una sola etiqueta, sino de tres elementos: la confianza del clasificador, el soporte visual de YOLO y la compatibilidad de los rasgos con una firma morfológica.

Los aciertos se concentraron en clases donde la firma morfológica fue clara. En los mosquitos, la combinación de `one_pair`, `membranous`, `piercing_sucking` y `slender` activó de forma consistente `signature_mosquito`. En las abejas ocurrió algo parecido con rasgos como `two_pairs`, `membranous`, `chewing` y `narrow_waist`. En las mariposas, los rasgos `scaly`, `siphoning` y `clavate` ayudaron a distinguir la clase.

Los errores y rechazos muestran dos patrones. El primero es la salida de revisión, donde el sistema prefirió no decidir porque faltó confianza o soporte visual. El segundo patrón es la

confusión entre firmas parecidas. El caso más claro fue `mantis_religiosa_000199`, donde los rasgos activaron `signature_grasshopper` y el sistema concluyó `grasshopper` en lugar de `mantis`.

En síntesis, el agente decide en dos niveles. Primero valida si la evidencia visual es aceptable. Después verifica si los rasgos forman una firma coherente con alguna clase. Si ambas condiciones se cumplen, emite clase final y, cuando aplica, especie final. Si no, marca revisión. Esta lógica permite que la decisión sea trazable y no solamente una salida directa de una red neuronal.

7. Limitaciones

La principal limitación es el tamaño del dataset. Al tener pocas imágenes, el sistema puede no generalizar bien a imágenes muy distintas. Además, algunas clases de YOLO son difíciles de detectar por su tamaño o ambigüedad visual, especialmente `mouthpart` y `legs`.

Otra limitación es que algunos rasgos de MobileNet se derivan de la especie, por lo que el modelo puede aprender asociaciones fuertes entre especie y rasgo. También existe dependencia del formato JSON: si cambian los nombres de campos o la estructura interna, el script de integración puede generar hechos incompletos.

DeepProbLog depende de la calidad del módulo perceptual. Si MobileNet predice mal los rasgos o si YOLO no detecta partes relevantes, la inferencia simbólica también se degrada. Además, los umbrales usados en las reglas fueron definidos manualmente, por lo que el desempeño puede variar si se modifica la configuración.

Finalmente, esta implementación usa evidencia ya calculada por YOLO y MobileNet. No se entrenaron predicados `nn(...)` directamente dentro de DeepProbLog. Esto simplifica la integración, pero todavía no representa una solución neuro-simbólica de extremo a extremo.

8. Conclusiones

El proyecto logró integrar visión por computadora y razonamiento lógico-probabilístico en un agente neuro-simbólico. YOLO permitió detectar partes del insecto, MobileNet clasificó especies y rasgos desde la imagen completa, y DeepProbLog permitió razonar sobre la evidencia generada.

El sistema cumple con los requisitos principales porque recibe un estado, genera una acción, implementa una lógica de decisión clara y permite evaluar su comportamiento con diferentes imágenes. Aunque existen limitaciones por el tamaño del dataset, la dificultad de detectar partes pequeñas y la dependencia de reglas manuales, la arquitectura propuesta permite explicar mejor la decisión final que un modelo puramente neuronal.

La arquitectura final debe entenderse como una integración modular: YOLO y MobileNet no dependen uno del otro durante la predicción, sino que generan evidencias complementarias. La decisión final ocurre cuando DeepProbLog combina esas evidencias a partir del JSON generado.

9. Enlaces de entrega

Como parte de la entrega final del proyecto, se incluyen el repositorio del código fuente y el video demostrativo del sistema. El repositorio contiene los códigos, archivos de integración, modelos, reglas lógicas y documentación necesaria para reproducir el proyecto. El video muestra el problema, el funcionamiento del agente, la demo del sistema, los resultados obtenidos y la conclusión.

Tabla 9: Material complementario de entrega.

Recurso	Enlace
Video demostrativo	https://youtu.be/mIC20bk7Do0
Repositorio del proyecto	https://github.com/BTG47/insect-classification-with-prolog-CIP

Referencias

- [1] Manhaeve, R., Dumančić, S., Kimmig, A., Demeester, T., & De Raedt, L. (2021). *Neural probabilistic logic programming in DeepProbLog*. Artificial Intelligence, 298, 103504. <https://doi.org/10.1016/j.artint.2021.103504>
- [2] De Raedt, L., Kimmig, A., & Toivonen, H. (2007). *ProbLog: A probabilistic Prolog and its application in link discovery*. Proceedings of the 20th International Joint Conference on Artificial Intelligence, 2468–2473. <https://www.ijcai.org/Proceedings/07/Papers/396.pdf>
- [3] Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). *You Only Look Once: Unified, Real-Time Object Detection*. IEEE Conference on Computer Vision and Pattern Recognition, 779–788. https://openaccess.thecvf.com/content_cvpr_2016/html/Redmon_You_Only_Look_CVPR_2016_paper.html
- [4] Howard, A. G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Andreetto, M., & Adam, H. (2017). *MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications*. arXiv. <https://arxiv.org/abs/1704.04861>
- [5] Bray, T. (2017). *The JavaScript Object Notation (JSON) Data Interchange Format*. RFC 8259. Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc8259>
- [6] d’Avila Garcez, A., Besold, T. R., De Raedt, L., Földiák, P., Hitzler, P., Icard, T., Kühnberger, K.-U., Lamb, L. C., Miikkulainen, R., & Silver, D. L. (2015). *Neural-Symbolic Learning and Reasoning: Contributions and Challenges*. AAAI Spring Symposium Series. <https://cdn.aaai.org/ocs/10281/10281-45212-1-PB.pdf>
- [7] Ultralytics. YOLO Documentation. <https://docs.ultralytics.com/>
- [8] PyTorch. PyTorch Documentation. <https://pytorch.org/docs/stable/index.html>